



AUDIT REPORT

February 2026

For

WAVE

The logo for WAVE, featuring the word "WAVE" in a bold, white, sans-serif font. Below the "W" and "A" is a stylized graphic of three blue waves.

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	08
Techniques and Methods	10
Types of Severity	12
Types of Issues	13
Severity Matrix	14
 High Severity Issues	15
1. Incorrect cliff calculation could lead to early vesting unlock	15
 Medium Severity Issues	16
2. Lack of Freshness Check on Oracle Price Feeds	16
3. Consider using safeTransfer/safeTransferFrom instead	17
 Low Severity Issues	18
4. Consider using Ownable2Step instead	18
5. lastClaimTime Precision Loss Causes Cumulative Rounding	19
6. CLIFF_INTERVAL = 2 Minutes Is a Testnet Value	20
7. remainingAmount Overwritten on Subsequent Purchase	21
 Informational Issues	22
5. Redundant SafeMath Use	22



Automated Tests	23
Functional Tests	23
Threat Model	24
Closing Summary & Disclaimer	29



Executive Summary

Project Name	OneWave
Protocol Type	Token, Token Presale/Vesting
Project URL	https://www.onewave.app/
Overview	The WavePresale contract manages the protocol's fundraising and token distribution. It facilitates a three-round presale structure, allowing users to purchase tokens using BNB, USDT, or USDC, with exchange rates determined dynamically via Chainlink Price Feeds. The contract handles the post-sale distribution logic, enforcing a vesting schedule that includes an initial TGE (Token Generation Event) release followed by a linear, time-based cliff vesting period for the remaining allocation. The WaveTokenV2 contract is a standard ERC20 implementation representing the protocol's new utility token with a fixed total supply. Beyond standard transfer functionality, it includes a dedicated migration mechanism that allows holders of the legacy "Old Wave" token to swap their assets for V2 tokens. This migration process atomically burns the legacy tokens to reduce their supply while transferring the equivalent amount of V2 tokens to the user.
Audit Scope	The scope of this Audit was to analyze the OneWave Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Maxtron-Private-Services-Limited/wave-smart-contract-audit
Branch	main
Contracts in Scope	WavePresale.sol, WaveTokenV2.sol
Commit Hash	99b5b28c1d329dd645f3b18665d9202572e1bad3
Language	Solidity
Blockchain	BNB Smart Chain, EVM
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	13th February 2026



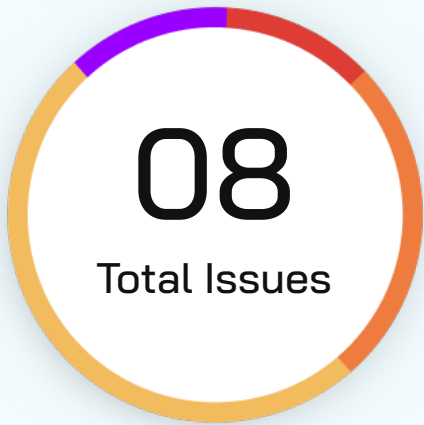
Updated Code Received	13th February 2026 and 21st Feb 2026
Review 2	23rd February 2026
Fixed In	0751dc939784d8c99bcfa2ddf36ac031844a0889

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	1 (12.5%)
Medium	2 (25.0%)
Low	4 (50.0%)
Informational	1 (12.5%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	1	2	4	1



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Incorrect cliff calculation could lead to early vesting unlock	High	Resolved
2	Lack of Freshness Check on Oracle Price Feeds	Medium	Resolved
3	Consider using safeTransfer/ safeTransferFrom instead	Medium	Resolved
4	Consider using Ownable2Step instead	Low	Resolved
5	lastClaimTime Precision Loss Causes Cumulative Rounding	Low	Resolved
6	CLIFF_INTERVAL = 2 Minutes Is a Testnet Value	Low	Resolved
7	remainingAmount Overwritten on Subsequent Purchase	Low	Resolved
8	Redundant SafeMath Use	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



High Severity Issues

Incorrect cliff calculation could lead to early vesting unlock

Resolved

Path

WavePresale.sol

Function Name

`cliffClaim(uint256 round)`

Description

The vesting logic distributes tokens per interval using:
`uint256 claimAmount = purchase.totalTokens.div(12).mul(intervals);`
However, the vesting pool after TGE is not `totalTokens` – it is:
`remainingTokens = totalTokens - tgeAmount`

Because the contract calculates each vesting release using the full allocation instead of the post-TGE allocation, users unlock tokens faster than the intended schedule. The final vesting portion is clamped by `remainingAmount`, causing the vesting duration to shrink instead of lasting the full planned period.

```
1  uint256 claimAmount = purchase.totalTokens.div(12).mul(intervals);
```

Impact

Vesting finishes earlier than advertised

Likelihood: High

POC

Consider a user purchasing 1,200 Tokens with a 20% TGE.

TGE Claim: User claims 20% (240 tokens).

`remainingAmount = 960 tokens.`

Vesting Month 1:

Intended Behavior: $960 \text{ remaining} / 12 \text{ months} = 80 \text{ tokens/month}$.

Actual Code Behavior: $1200 \text{ total} / 12 \text{ months} = 100 \text{ tokens/month}$.

Result:

The user receives 20 extra tokens per month.

After 9.6 months ($960 / 100$), the user's `remainingAmount` reaches 0.

The vesting schedule effectively ends at Month 10, rendering the final 2 months of the "12-month cliff" irrelevant.

Recommendation

Adjust the vesting formula to calculate the monthly unlock based on the remainder (Total - TGE).



Medium Severity Issues

Lack of Freshness Check on Oracle Price Feeds

Resolved

Path

WavePresale.sol

Function Name

getBNBPrice, getUSDCPrice, getUSDTPrice

Description

The contract utilizes Chainlink's latestRoundData() to fetch the prices of BNB, USDT, and USDC. However, the implementation only validates that the returned price is greater than zero. It fails to check the updatedAt timestamp or the answeredInRound ID.

```
1  function getBNBPrice() public view returns (uint256) {
2      (, int256 price, , ,) = bnbPriceFeed.latestRoundData();
3      require(price > 0);
4      return uint256(price);
5  }
6
7  function getUSDTPrice() public view returns (uint256) {
8      (, int256 price, , ,) = usdtPriceFeed.latestRoundData();
9      require(price > 0);
10     return uint256(price);
11 }
12
13 function getUSDCPrice() public view returns (uint256) {
14     (, int256 price, , ,) = usdcPriceFeed.latestRoundData();
15     require(price > 0);
16     return uint256(price);
17 }
18
```

If the Chainlink aggregator pauses, goes offline, or returns stale data (e.g., during periods of extreme network congestion or a "black swan" market crash), the contract will continue to calculate token amounts based on the frozen, outdated price.

Impact

Incorrect Pricing followed by potential arbitrage.

Likelihood: Medium



POC

State: The Oracle reports BNB = \$300. updatedAt is T.

Event: BNB market price crashes to \$100.

Failure: The Oracle stops updating due to congestion or maintenance. It still returns \$300 (stale price).

Exploit: A user buys with 10 BNB (Real value: \$1,000).

Result: The contract calculates the token amount based on \$300/BNB (perceived value: \$3,000).

Outcome: The user receives 3x more tokens than they are entitled to

Recommendation

Implement a "staleness" check to ensure the price data is recent. The threshold (heartbeat) should ideally be configurable or set according to the specific asset's Chainlink parameters

Consider using safeTransfer/safeTransferFrom instead**Resolved****Path**

WavePresale.sol

Description

The contract uses the standard IERC20.transfer and transferFrom functions which expect a boolean return value.

However in case of non standard ERC20 tokens that does not return boolean value, this might pose a threat to the system.

Impact

Potential stuck funds or loss of funds with non compliant ERC20's

Likelihood: Medium to Low

Recommendation

Consider using safeTransfer from OpenZeppelin in the code



Low Severity Issues

Consider using Ownable2Step instead

Resolved

Path

WavePresale.sol, WaveTokenV2.sol

Description

The contracts inherit from the standard OpenZeppelin Ownable contract, which allows ownership to be transferred in a single transaction via `transferOwnership(newOwner)`.

If the admin accidentally types the wrong address (e.g., a typo or a contract address without a write function) or sends ownership to a “dead” address, access to critical administrative functions (like `withdrawFunds`, `startPresaleRound`, `setRoundTGETime`) will be permanently lost.

Impact

Permanent loss of control over protocol’s critical functions

Likelihood: Low

Recommendation

Use Ownable2Step instead of Ownable. This implements a “commit-confirm” pattern where the new owner must explicitly accept the ownership transfer, preventing accidental loss.



lastClaimTime Precision Loss Causes Cumulative Rounding**Resolved****Path**

WavePreSale.sol

Function Name

cliffClaim()

Description

At line 342, lastClaimTime is set to block.timestamp rather than the aligned interval boundary. Any sub-interval remainder is lost. Over 12 claims, this cumulative loss can shift the final claim by up to 12 * (CLIFF_INTERVAL - 1) seconds. With production CLIFF_INTERVAL = 30 days, users could lose up to ~12 days of vesting attribution.

```
// WavePresale.sol:329-342
require(block.timestamp >= purchase.lastClaimTime.add(CLIFF_INTERVAL));

uint256 intervals =
(block.timestamp.sub(purchase.lastClaimTime)).div(CLIFF_INTERVAL);
uint256 claimAmount = purchase.totalTokens.div(12).mul(intervals);
// ...
purchase.lastClaimTime = block.timestamp; // @audit should align to interval
boundary
```

Recommendation

```
// Align to interval boundary to prevent precision loss
purchase.lastClaimTime =
purchase.lastClaimTime.add(intervals.mul(CLIFF_INTERVAL));
```



CLIFF_INTERVAL = 2 Minutes Is a Testnet Value

Resolved

Path

WavePreSale.sol

Description

CLIFF_INTERVAL is set to 2 minutes as a constant at line 112. With 12 installments, all cliff vesting completes in 24 minutes (12 * 2 min) instead of the intended 12 months. As a Solidity constant, it is compiled into the bytecode and cannot be changed after deployment. Up to 75M presale tokens across all 3 rounds would fully unlock within 24 minutes of the cliff period starting.

```
// WavePresale.sol:112
uint256 public constant CLIFF_INTERVAL = 2 minutes; // @audit Should be 30 days
for production

// WavePresale.sol:331-332 - Used in cliff vesting calculation
uint256 intervals =
(block.timestamp.sub(purchase.lastClaimTime)).div(CLIFF_INTERVAL);
uint256 claimAmount = purchase.totalTokens.div(12).mul(intervals);
// With CLIFF_INTERVAL = 2 min, after 24 minutes: intervals = 12, claimAmount =
totalTokens
```

Recommendation

```
uint256 public constant CLIFF_INTERVAL = 30 days;
Or
make it configurable per-round if different vesting cadences are needed:
uint256 public cliffInterval = 30 days;

function setCliffInterval(uint256 _interval) external onlyOwner {
    require(_interval >= 1 days && _interval <= 365 days, "Invalid interval");
    cliffInterval = _interval;
}
```



remainingAmount Overwritten on Subsequent Purchase

Resolved

Path

WavePreSale.sol

Description

In the WavePresale contract, when a buyer makes multiple purchases, the new purchase's token amount overwrites the previously calculated remainingAmount instead of being added to it. In code, each purchase does:

```
purchase.totalTokens = purchase.totalTokens.add(tokenAmount) ;  
purchase.remainingAmount = purchase.totalTokens; // BUG: resets remainingAmount
```

This logic ignores any tokens the user has already claimed (TGE or cliff vesting).

The intended invariant is $\text{remainingAmount} == \text{totalTokens} - \text{claimedTGE} - \text{claimedCliff}$, but the code resets remainingAmount to totalTokens on each purchase.

As a result of this bug, if a user makes a second purchase after already claiming some tokens, their remainingAmount is set too high. This can cause later vesting claims to exceed the allowed limit and revert.

```
// After a purchase  
purchase.totalTokens = purchase.totalTokens.add(tokenAmount) ;  
purchase.remainingAmount = purchase.totalTokens; // ← BUG: previous claims  
ignored
```

Example

Round 1 sale, user buys 1000 tokens. $\Rightarrow$ totalTokens=1000, remainingAmount=1000, claimedTGE=0.

After TGE, user claims 100 tokens (10% of 1000) $\Rightarrow$ claimedTGE=100, remainingAmount=900.

User buys 500 more tokens. Code sets totalTokens=1500, but also wrongly sets remainingAmount=1500 (should be 1400).

After 12 cliff intervals (all tokens vest), the user tries to claim. The contract computes claimAmount=1500 and checks $100 \text{ (TGE)} + 0 + 1500 \leq 1500$, which fails, causing a revert.

If the user tries at 11 intervals: claimAmount=1375, which passes, leaving claimedCliff=1375 and remainingAmount=125.

Next interval: claimAmount=125 but the check $100 + 1375 + 125 \leq 1500$ fails. Hence 125 tokens are stuck in the contract forever.

Recommendation

```
purchase.totalTokens = purchase.totalTokens.add(tokenAmount);  
purchase.remainingAmount = purchase.remainingAmount.add(tokenAmount);
```

This preserves the previously claimed amounts and maintains $\text{remainingAmount} = \text{totalTokens} - \text{claimedTGE} - \text{claimedCliff}$.



Informational Issues

Redundant SafeMath Use

Resolved

Path

WavePreSale.sol

Description

he contract utilizes the SafeMath library for arithmetic operations. However, the contract uses Solidity version 0.8.20.

Since Solidity 0.8.0, arithmetic overflow/underflow checks are built directly into the language. Using SafeMath wrapper functions in versions 0.8+ is redundant and increases gas costs because each arithmetic operation incurs the overhead of an external library function call (or internal jump).

Recommendation

Consider removing safeMathLibrary



Functional Tests

- ✓ Should revert if the current time is before `startTime` or after `endTime`.
- ✓ Should revert if the purchase amount exceeds the Round's `totalAllocation`
- ✓ First claim should revert if called before the `tgeTime`
- ✓ Should update `lastClaimTime` to the current block timestamp after a successful claim
- ✓ Should transfer the exact matching amount of New Tokens to the user.
- ✓ Should correctly initialize `userPurchases` for multiple addresses in a single transaction.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20	Interface	Interaction with USDT, USDC, and WaveToken	Tokens follow standard ERC20 behavior (return bools, fire events)	USDT (ETH) Risk: Non-standard tokens (return void) will cause reverts.
AggregatorV3Interface	Oracle	Fetching BNB/USD, USDT/USD, USDC/USD prices	Chainlink feeds are live, accurate, and not manipulated	Stale Data: If feed freezes, users can buy tokens at wrong prices (Arbitrage).
SafeMath	Library	Arithmetic operations (add, sub, mul, div)	Correct math logic (though redundant in Sol 0.8+)	Gas inefficiency; False sense of security if logic is flawed elsewhere.
IOldWave	Interface	Interacting with the old token during migration	Old token implements burn(uint256) and returns bool	Signature Mismatch: If old token burn returns void, migration reverts.
OpenZeppelin (Ownable)	Abstract	Access control for admin functions	Only the deployed address allows critical state changes	Single Key Risk: Compromised owner key leads to total protocol loss (rug pull).



2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
WavePresale.sol	buyWithBNB()	What this function can do Accept BNB, update userPurchases, increase soldAmount. What this function cannot/should not do Sell more than totalAllocation; Accept funds if round is inactive. Main invariant(s) soldAmount <= totalAllocation Access level Public
WavePresale.sol	buyWithUSDC/ buyWithUSDT	What this function can do Transfer tokens from User to Contract, update state. What this function cannot/should not do Succeed without user approval; Handle "fee-on-transfer" tokens correctly (balance mismatch). Main invariant(s) soldAmount <= totalAllocation Access level Public



Contract Name	Function	Category
WavePresale.sol	tgeClaim()	What this function can do Set the TGE timestamp for a round. What this function cannot/should not do Set time in the past; Set time before round ends. Main invariant(s) tgeTime > endTime Access level Owner Controlled
WaveTokenV2.sol	migrate()	What this function can do Burn Old Tokens from user, Send New Tokens to user. What this function cannot/should not do Succeed if V2 contract has no liquidity; Work without allowance. Main invariant(s) 1 Old Token = 1 New Token Access level Public



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets	Centralization Risk
WavePresale	Native (BNB)	User Presale Payments	High Risk: Primary target for hacks. Holds all raised BNB.
WavePresale	ERC20 (USDT/USDC)	User Presale Payments	High Risk: Holds all stablecoin revenue.
WavePresale	ERC20 (WaveToken)	Unclaimed User Allocations	Must hold enough tokens to cover all claims (TGE + Vesting).
WaveTokenV2	ERC20 (WaveV2)	Migration Liquidity	Critical Bug Must hold supply to function.



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
WavePresale	buyWithBNB	BNB	—	Public	Price Oracle: Dependent on Chainlink. Flash
WavePresale	buyWithUSDT	USDT	—	Public	Fake Token: If malicious token passed (though hardcoded address prevents this).
WavePresale	tgeClaim	—	WaveToken	Public	Timing: If tgeTime set wrong, funds lock or release early.
WavePresale	cliffClaim	—	WaveToken	Public	Math Bug: Accelerated vesting (110% bug) drains reserve early.
WavePresale	withdrawFunds	—	BNB/USDT	Owner	Rug Pull: Owner can drain 100% of raised funds at any time.
WaveTokenV2	migrate	OldWave (Burned)	WaveV2	Public	DOS: Fails if contract has no V2 tokens.



Closing Summary

In this report, we have considered the security of OneWave. We performed our audit according to the procedure described above.

One high, two medium, four low and one informational issues were found. The OneWave Team resolved all the issues mentioned.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For

WAVE



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com